

Redis SQL Trino

Version 0.4.0-SNAPSHOT

Table of Contents

Introduction.....	1
Installation.....	2
Configuration.....	3
SQL support	5
Clients	8
Build.....	9
Complete Walkthrough.....	10

Introduction

Redis SQL Trino is a [Trino](#) connector which allows access to Redisearch data from Trino.

This guide provides documentation and usage information across the following topics:

- [Installation](#)
- [Configuration](#)
- [SQL support](#)
- [Clients](#)
- [Walkthrough](#)

Installation

Trino

Trino installation instructions are available at <https://trino.io/docs/current/installation.html>.

RediSearch connector

Download [latest release](#) and unzip without any directory structure under:

`<trino>/plugin/redisearch`

Create a RediSearch connector configuration file and change/add [properties](#) as needed.

Configuration

To configure the RediSearch connector, create a catalog properties file and change/add properties as needed.

etc/catalog/redisearch.properties

```
connector.name=redisearch
redisearch.uri=redis://localhost:6379
```

Table 1. Connector properties

Property name	Description	Default
<code>redisearch.default-schema-name</code>	The schema that contains all tables defined without a qualifying schema name.	<code>default</code>
<code>redisearch.case-insensitive-names</code>	Match index names case insensitively.	<code>false</code>
<code>redisearch.cursor-count</code>	Number of rows read during each aggregation cursor fetch.	<code>1000</code>
<code>redisearch.table-cache-refresh</code>	Seconds an index's schema is cached. Indexes and fields created outside Trino can take this long to appear.	<code>60</code>

Table 2. Redis connection properties

Property name	Description	Default
<code>redisearch.uri</code>	A Redis connection string. Redis URI syntax .	
<code>redisearch.username</code>	Redis connection username.	
<code>redisearch.password</code>	Redis connection password.	
<code>redisearch.cluster</code>	Connect to a Redis Cluster.	<code>false</code>
<code>redisearch.resp2</code>	Force Redis protocol version to RESP2.	<code>false</code>

The RediSearch connector provides additional security options to support Redis servers with TLS mode.

Table 3. TLS properties

Property name	Description	Default
<code>redisearch.insecure</code>	Allow insecure connections (e.g. invalid certificates) when using SSL.	<code>false</code>

Property name	Description	Default
<code>redisearch.cacert-path</code>	X.509 CA certificate file to verify with.	
<code>redisearch.key-path</code>	PKCS#8 private key file to authenticate with (PEM format).	
<code>redisearch.key-password</code>	Password of the private key file, or null if it's not password-protected.	
<code>redisearch.cert-path</code>	X.509 certificate chain file to authenticate with (PEM format).	

SQL support

Tables and columns

Each Redis Query Engine index is a table in the `default` schema (see `redisearch.default-schema-name`). A table has these columns:

- One per indexed field, named by its attribute (the `AS` name for JSON paths).
- One per field that isn't indexed but appears in the first 10 documents returned by `FT.SEARCH <index> *`.
- For JSON indexes, a `$` column holding the whole document as JSON text.
- A hidden `key` column holding the document's Redis key. Hidden columns are left out of `SELECT *, so name it: SELECT key, * FROM beers`.

Table 4. Type mapping

Redis field type	Trino type
NUMERIC	DOUBLE
TAG, TEXT, GEO, GEOSHAPE, VECTOR	VARCHAR
Not indexed	VARCHAR

Statements

Table 5. Supported statements

Statement	Notes
<code>SELECT</code>	All of Trino's query syntax, including joins (also across catalogs), subqueries, <code>WITH</code> , <code>UNION</code> , <code>DISTINCT</code> , <code>GROUP BY</code> , <code>ORDER BY</code> and window functions. Pushdown describes which parts run in Redis.
<code>SHOW SCHEMAS</code> , <code>SHOW TABLES</code> , <code>SHOW COLUMNS</code> , <code>DESCRIBE</code> , <code>SHOW CREATE TABLE</code> , <code>information_schema</code>	<code>SHOW TABLES</code> lists the indexes returned by <code>FT._LIST</code> .
<code>EXPLAIN</code>	The table scan shows the filters, limit and aggregations sent to Redis.
<code>CREATE TABLE</code>	Creates a hash index named after the table, with key prefix <code><table>:.</code> See column types . <code>IF NOT EXISTS</code> is supported.
<code>CREATE TABLE AS SELECT</code>	BOOLEAN and DATE values are written but not indexed (#43).
<code>INSERT</code> , <code>INSERT ... SELECT</code>	Hash indexes only (#44). Each row is a new hash with key <code><prefix><ULID></code> , where <code><prefix></code> is the index's first key prefix.
<code>UPDATE</code>	Hash indexes only. Setting a column to <code>NULL</code> removes the field from the hash.
<code>DELETE</code>	Deletes the matching keys, on hash and JSON indexes.

Statement	Notes
MERGE	Hash indexes only. A WHEN NOT MATCHED THEN INSERT column list must include the table's last column (#42).
ALTER TABLE ... ADD COLUMN	Adds a field to the index with FT.ALTER . Columns are always added at the end.
DROP TABLE	Drops the index and deletes its documents (FT.DROPINDEX ... DD).

These statements fail with a **This connector does not support ...** error:

- CREATE SCHEMA, DROP SCHEMA, ALTER SCHEMA
- CREATE OR REPLACE TABLE
- ALTER TABLE ... RENAME TO, DROP COLUMN, RENAME COLUMN, ALTER COLUMN ... SET DATA TYPE
- TRUNCATE
- COMMENT ON
- CREATE VIEW, CREATE MATERIALIZED VIEW
- Queries on table versions (**FOR VERSION AS OF**, **FOR TIMESTAMP AS OF**)

Writes also fail when fault-tolerant execution is enabled (**retry-policy** other than **NONE**).

Table 6. Column types in CREATE TABLE

Trino type	Index field	Read back as
BIGINT, INTEGER, SMALLINT, TINYINT, DOUBLE, REAL, DECIMAL	NUMERIC	DOUBLE
BOOLEAN, DATE, TIMESTAMP(3), TIMESTAMP(3) WITH TIME ZONE	NUMERIC	DOUBLE. Timestamps are stored as epoch milliseconds.
VARCHAR, CHAR	TAG	VARCHAR
UUID	TAG	VARCHAR. CREATE TABLE AS SELECT with UUID values fails (#43).

Other types, such as ARRAY, MAP, ROW, VARBINARY and TIME, are rejected. Because columns are read back from the index, a column declared as BIGINT or BOOLEAN becomes DOUBLE, and later **INSERT** statements must supply numbers. VARCHAR columns become TAG fields, which match exactly and support **LIKE** but not full-text search.

Pushdown

Redis evaluates these parts of a query. Trino evaluates everything else on the rows Redis returns.

- Filters on indexed fields, combined with **AND**:
 - = and **IN**. TAG fields match exactly. TEXT fields use full-text matching (#46).
 - <, <=, >, >= and **BETWEEN** on NUMERIC fields.

- **LIKE** on TAG and TEXT fields, as a wildcard query. Trino rewrites prefix patterns such as 'abc%' into a range, which it evaluates itself.
- **count(*)**, **min**, **max**, **sum** and **avg** on numeric columns, with or without **GROUP BY**, as **FT.AGGREGATE** with **GROUPBY** and **REDUCE**. If any aggregate in a query can't be pushed down, such as **count(DISTINCT ...)**, Trino computes all of them.
- **LIMIT**. Trino sorts **ORDER BY ... LIMIT** itself.

Known limitations

These queries return wrong results or fail.

Problem	Workaround	Issue
IS NULL and IS NOT NULL on an indexed field return every row.		#39
count(column) counts every row, including those where the column is null.	count_if(column IS NOT NULL)	#45
= and IN on a TEXT field match any document containing the words, not just equal values.	Index the field as TAG for exact matches.	#46
<> , < , > and BETWEEN on an indexed VARCHAR column fail with Range constraint not supported .		#40
HAVING , or a filter on a subquery's aggregate, fails with Unknown field ... syntheticColumn0 when the aggregation is pushed down.		#41
MERGE ... WHEN NOT MATCHED THEN INSERT fails when the column list leaves out the table's last column.	List every column and pass NULL for the ones you don't have.	#42
CREATE TABLE AS SELECT with BOOLEAN or DATE values succeeds, but those rows can't be queried.	Cast them first, e.g. CAST(flag AS integer) or CAST(day AS varchar) .	#43
INSERT into a JSON index writes a hash the index never sees. UPDATE and MERGE on a JSON index fail.		#44
A scan without a SQL LIMIT stops at redisearch.default-limit documents (10,000 by default).	Raise redisearch.default-limit .	#35
Queries return partial results while an index is still being built.	Wait for FT.INFO to report indexing as 0 .	#36

Clients

JDBC Driver

The Trino JDBC driver allows users to access Trino from Java-based applications, and other non-Java applications running in a JVM.

Refer to the [Trino documentation](#) for setup instructions.

The following is an example of a JDBC URL used to create a connection to Redis SQL Trino:

```
jdbc:trino://example.net:8080/redisearch/default
```

Tableau

Refer to the [Tableau documentation](#) for setup instructions.

Trino CLI

Refer to the [Trino CLI documentation](#) for setup instructions.

Build

Run these commands to build the Trino connector for Redisearch from source (requires Java 25+):

```
git clone https://github.com/redis-field-engineering/redis-sql-trino.git
cd Redis SQL Trino
./mvnw clean package -DskipTests
```

Complete Walkthrough

Follow these step-by-step instructions to deploy a single-node Trino server on Ubuntu.

Install Java

Trino requires a 64-bit version of Java 25. It is recommended to use [Azul Zulu](#) as the JDK.

```
$ java -version
openjdk version "25.0.1" 2025-10-21 LTS
OpenJDK Runtime Environment Zulu25.30+17-CA (build 25.0.1+8-LTS)
OpenJDK 64-Bit Server VM Zulu25.30+17-CA (build 25.0.1+8-LTS, mixed mode, sharing)
```

Set up Trino

Download the Trino server tarball and unpack it.

```
wget https://repo1.maven.org/maven2/io/trino/trino-server/483/trino-server-483.tar.gz
mkdir /usr/lib/trino
tar xzvf trino-server-483.tar.gz --directory /usr/lib/trino --strip-components 1
```

Trino needs a data directory for storing logs, etc. It is recommended to create a data directory outside of the installation directory, which allows it to be easily preserved when upgrading Trino.

Create a data directory

```
mkdir -p /var/trino
```

Create an **etc** directory inside the installation directory to hold configuration files.

```
mkdir /usr/lib/trino/etc
```

Create a **node.properties** file.

/usr/lib/trino/etc/node.properties

```
node.environment=production
node.id=ffffffff-ffff-ffff-ffff-ffffffffffffff
node.data-dir=/var/trino
```

Create a JVM config file.

/usr/lib/trino/etc/jvm.config

```
-server
-Xmx16G
-XX:InitialRAMPercentage=80
-XX:MaxRAMPercentage=80
-XX:G1HeapRegionSize=32M
-XX:+ExplicitGCInvokesConcurrent
-XX:+ExitOnOutOfMemoryError
-XX:+HeapDumpOnOutOfMemoryError
-XX:-OmitStackTraceInFastThrow
-XX:ReservedCodeCacheSize=512M
-XX:PerMethodRecompilationCutoff=10000
-XX:PerBytecodeRecompilationCutoff=10000
-Djdk.attach.allowAttachSelf=true
-Djdk.nio.maxCachedBufferSize=2000000
-XX:+UnlockDiagnosticVMOptions
-XX:+UseAESCTRIinsics
```

Create a config properties file.

/usr/lib/trino/etc/config.properties

```
coordinator=true
node-scheduler.include-coordinator=true
http-server.http.port=8080
discovery.uri=http://localhost:8080
```

Create a logging configuration file.

/usr/lib/trino/etc/log.properties

```
io.trino=INFO
```

Set up Redis SQL Trino

Download latest [release](#) and unzip without any directory structure under **plugin/redisearch**:

```
wget https://github.com/redis-field-engineering/redis-sql-trino/releases/download/v0.4.0-SNAPSHOT/redis-sql-trino-0.4.0-SNAPSHOT.zip
unzip -j redis-sql-trino-0.4.0-SNAPSHOT.zip -d /usr/lib/trino/plugin/redisearch
```

Create a etc/catalog subdirectory:

```
mkdir /usr/lib/trino/etc/catalog
```

Create a Redisearch connector configuration file:

/usr/lib/trino/etc/catalog/redisearch.properties

```
connector.name=redisearch
redisearch.uri=redis://localhost:6379
```

Change and/or add [properties](#) as needed.

Start Trino

Start the Trino server:

```
/usr/lib/trino/bin/launcher run
```

Run Trino CLI

Download and run [trino-cli-483-executable.jar](#):

```
wget https://repo1.maven.org/maven2/io/trino/trino-cli/483/trino-cli-483-
executable.jar -O /usr/local/bin/trino
chmod +x /usr/local/bin/trino
trino --catalog redisearch --schema default
```

Run a SQL query:

```
trino:default> select * from mySearchIndex;
```